

# Computer Vision

## Stereo (3D) & Augmented Reality

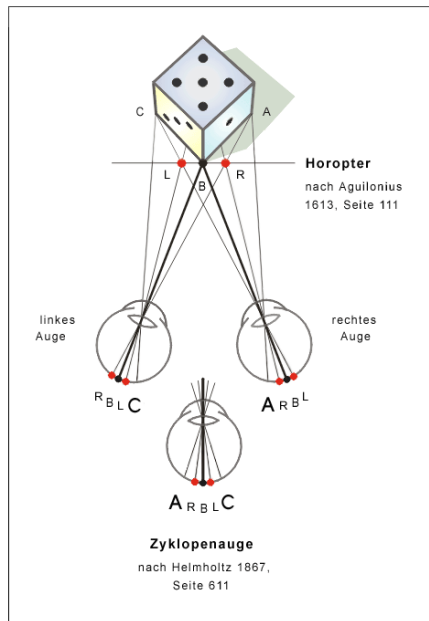
Alexandra Posekany

December 2025

# Stereo Vision & Kameramodelle - Sehen wie der Mensch

# Das "Zyklopen-Problem"

Wenn wir ein Foto machen,  
projizieren wir die 3D-Welt auf  
eine 2D-Ebene (Sensor).  
Informationsverlust: Wir verlieren  
die Tiefe ( $Z$ ).  
Ein großes Auto weit weg sieht  
genauso groß aus wie ein  
Spielzeugauto nah dran.  
Ziel der 3D-CV: Rekonstruktion  
von  $Z$  aus einem oder mehreren  
2D-Bildern.



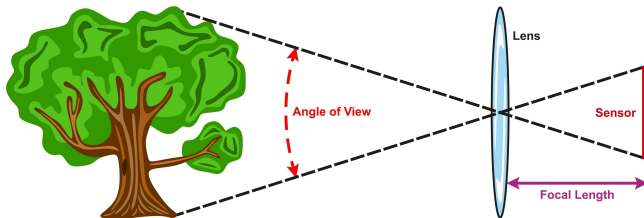
# Das Lochkameramodell (Pinhole Camera)

Das einfachste mathematische Modell einer Kamera: Lichtstrahlen fallen durch ein winziges Loch (Zentrum) auf die Bildebene.

Strahlensatz: Ähnliche Dreiecke. (Das Minuszeichen kommt vom Bild, das auf dem Kopf steht; mathematisch drehen wir es oft um).

$$x = -f \cdot \frac{X}{Z}, \quad y = -f \cdot \frac{Y}{Z}$$

Focal Length and Angle of View



The greater the focal length, the closer the image and the less the angle of view

The less the focal length, the farther the image and the greater the angle of view

# Intrinsische Parameter (Das Innenleben)

Wie kommen wir von Welt-Koordinaten (Meter) zu Pixel-Koordinaten?

Hängt von der Kamera ab: Brennweite ( $f_x, f_y$ ): Zoom-Faktor.

Hauptpunkt ( $c_x, c_y$ ): Wo trifft die optische Achse den Sensor (meist Bildmitte).

Die Kalibrierungsmatrix  $K$ :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

# Extrinsische Parameter (Die Lage im Raum)

Wo ist die Kamera in der Welt?

Rotation ( $R$ ): Wie ist die Kamera gedreht? ( $3 \times 3$  Matrix).

Translation ( $t$ ): Wo steht die Kamera? ( $3 \times 1$  Vektor).

Pipeline: Welt-Punkt  $\rightarrow$  [Rotation/Translation]  $\rightarrow$  Kamera-Punkt  
 $\rightarrow$  [Intrinsics]  $\rightarrow$  Pixel.

# Linsenverzerrung (Distortion)

Echte Linsen sind keine perfekten Löcher.

Radiale Verzerrung: Gerade Linien werden krumm (“Fischaugen” /  
Tonnenverzerrung).

Muss vor jeder 3D-Berechnung korrigiert werden (“Undistortion”).

# Zusammenfassung Kameramodell

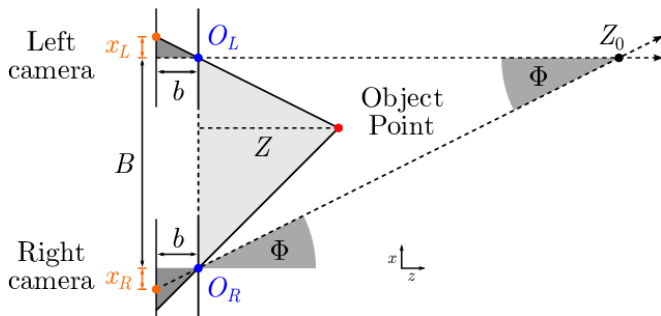
Ein Bild ist eine Projektion.

Um 3D zu messen, müssen wir die Kamera kalibrieren (Checkerboard-Muster), um Matrix  $K$  und Verzerrungskoeffizienten zu finden.



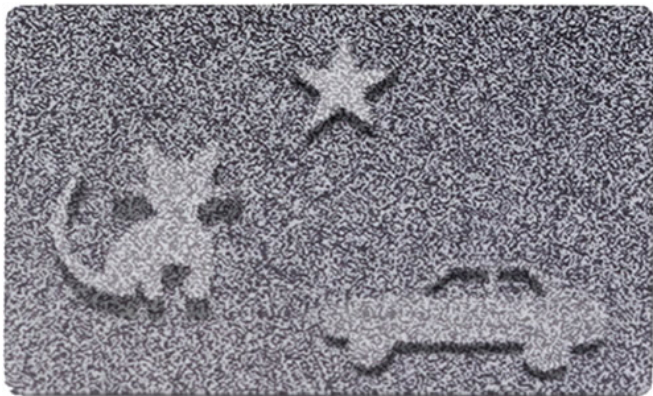
# Das Prinzip der Triangulation

Mit zwei Kameras können wir die Tiefe ( $Z$ ) berechnen, indem wir den Schnittpunkt der Sichtlinien bestimmen.



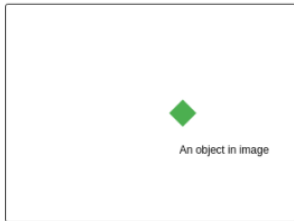
## Stereo sehen

Ein Punkt  $P$  in 3D wird links auf  $x_L$  und rechts auf  $x_R$  projiziert.

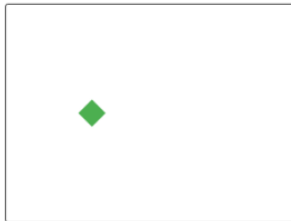


# Disparität

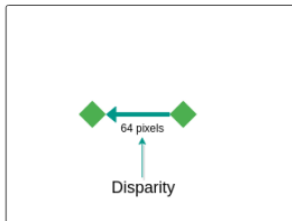
Left Image



Right Image



Left + Right Image



# Tiefenformel (Das Herzstück)

Herleitung über ähnliche Dreiecke (Strahlensatz).

$$Z = \frac{f \cdot B}{d}$$

Basisabstand ( $B$ ): Distanz zwischen den Kameras (Baseline).

Brennweite ( $f$ ): Fokussierung.

Interpretation:

- ▶ Tiefe  $Z$  ist umgekehrt proportional zur Disparität  $d$ .
- ▶ Um weit zu sehen, brauchen wir eine große Basis  $B$  (breitere Augen).

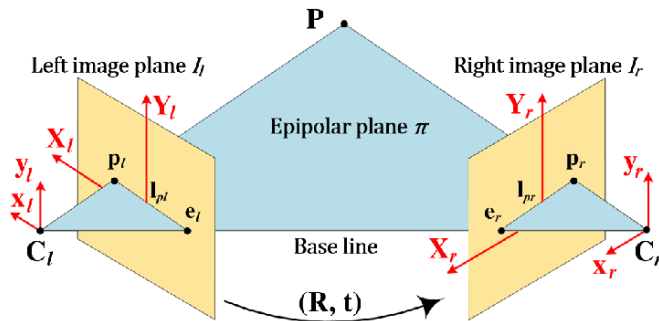
# Epipolare Geometrie (Das Such-Problem)

Woher wissen wir, welcher Pixel links zu welchem Pixel rechts gehört? (Matching).

Müssen wir das ganze rechte Bild durchsuchen? Nein!

Epipolarlinie: Ein Punkt im linken Bild korrespondiert zu einer Linie im rechten Bild.

Wir müssen nur entlang dieser Linie suchen (1D-Suche statt 2D-Suche).



# Stereo Matching Algorithmen

- ▶ Block Matching (SAD/SSD): Schiebe ein kleines Fenster entlang der Zeile und vergleiche Pixelwerte.

Um matching in ein Optimierungsproblem zu übersetzen, brauchen wir eine Tensorrepräsentation bzw. Matchingkosten.

- ▶ Sum of Squared-Differences (SSD)

$$SSD(A, B) = \sum_{i,j} (A_{ij} - B_{ij})^2$$

- ▶ Sum of absolute differences (SAD)  $SAD(A, B) = \sum_{i,j} |A_{ij} - B_{ij}|$

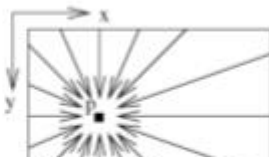
SAD robuster gegen noise/outliers

- ▶ Semi-Global Matching (SGM): Standard in OpenCV. Optimierte über das ganze Bild, um Sprünge und Rauschen zu glätten.

(a) Minimum Cost Path  $L_q(p, d)$



(b) 16 Paths from all Directions  $r$



# Disparity Map (Das Ergebnis)

Das Ergebnis von Stereo Vision ist kein 3D-Modell, sondern eine Disparity Map.

Ein Graustufenbild:

Hell (hohe Werte) = Nah.

Dunkel (niedrige Werte) = Fern.

Aus der Disparity Map berechnen wir dann die Punktwolke.

# Probleme von Stereo Vision

- ▶ **Texturlose Flächen:** Eine weiße Wand sieht links und rechts gleich aus → Kein Match möglich.
- ▶ **Wiederkehrende Muster:** Ein Zaun sieht überall gleich aus → Falsche Matches.
- ▶ **Verdeckungen (Occlusion):** Kamera links sieht etwas, das Kamera rechts nicht sieht (hinter einer Kante).



## Warum “Aktiv”?

Stereo Vision ist “passiv” (nutzt nur vorhandenes Licht).

Aktive Sensoren senden selbst Energie aus, um die Probleme von Stereo (Dunkelheit, keine Textur) zu lösen.

# Structured Light (Kinect v1)

Funktionsweise:

Ein Projektor wirft ein bekanntes Infrarot-Punktmuster auf die Szene.

Eine Infrarot-Kamera filmt das Muster.

**Tiefe:** Wenn das Muster auf ein Objekt trifft, wird es verzerrt. Aus der Verzerrung berechnet man die Tiefe.

Vorteil: Funktioniert auch auf weißen Wänden.

# Time-of-Flight (ToF) (Kinect v2, HoloLens)

Prinzip: 'Lichtgeschwindigkeit'  $c$  messen.

Kamera sendet Lichtpuls aus. Misst die Zeit  $\Delta t$ , bis er zurückkommt.

$$Distanz = \frac{c \cdot \Delta t}{2}$$

Modern: Phasenverschiebungsmessung (Genauer als Zeitmessung).

# LiDAR (Light Detection and Ranging)

Prinzip: Laser-Scanner.

Ein Laserstrahl tastet die Umgebung mechanisch ab (rotierender Spiegel) oder elektronisch (Solid State).

Einsatz: Autonomes Fahren (hohe Reichweite  $> 100\text{m}$ , extrem präzise).

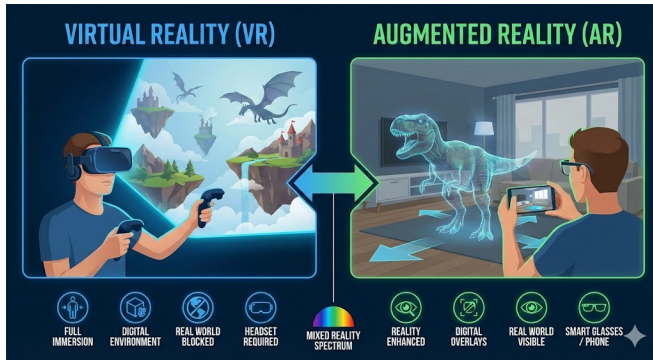
Output: Dichte, exakte Punktwolke.

# Vergleich der Technologien

| Tech             | Reichweite     | Genauigkeit  | Kosten                 | Problem                           |
|------------------|----------------|--------------|------------------------|-----------------------------------|
| Stereo           | Mittel         | Mittel       | Gering<br>(2 Web-cams) | Texturlose Flächen,<br>Dunkelheit |
| Structured Light | Nah (< 5m)     | Hoch         | Mittel                 | Sonnenlicht<br>(IR-Interferenz)   |
| ToF              | Mittel (< 10m) | Hoch         | Mittel/Hoch            | Reflektionen,<br>Multipath        |
| LiDAR            | Fern (> 100m)  | Sehr<br>Hoch | Hoch                   | Preis, Größe,<br>bewegliche Teile |

## Augmented Reality (AR)

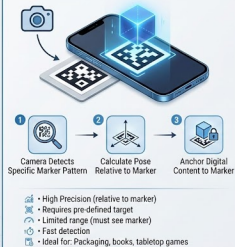
# Was ist Augmented Reality?



# Marker

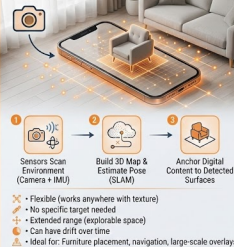
## AR TRACKING METHODS: MARKER-BASED VS. MARKERLESS

### MARKER-BASED AR (Fiducial Markers)



STABLE BUT CONSTRAINED

### MARKERLESS AR (SLAM / VIO)



FLEXIBLE BUT COMPLEX

TRADE-OFF: CONTROL VS. FREEDOM



# Die Transformationskette (Der Kern der AR)

Um ein virtuelles Objekt anzuzeigen, müssen wir durch vier Koordinatensysteme springen. Das sind reine Matrix-Multiplikationen.

1. Modell-Koordinaten: Das virtuelle Objekt selbst (z.B. ein 3D-Modell eines Autos, Ursprung in der Reifenmitte). Model Matrix ( $M$ )
2. Welt-Koordinaten: Wo steht das Auto im realen Raum? (z.B. auf dem Boden, 2 Meter vor mir). View Matrix (Extrinsics  $[R|t]$ )
3. Kamera-Koordinaten: Wo ist das Auto relativ zur Kameralinse? Projection Matrix (Intrinsics  $K$ )
4. Bild-Koordinaten (Pixel): Wo landet der Punkt auf dem 2D-Screen?

# Vom 3D-Punkt zum 2D-Pixel

Das ist die finale Formel, die die Grafik-Engine (z.B. Unity oder OpenGL) in jedem Frame millionenfach berechnet:

$$p_{pixel} = K \cdot [R_{cam}|t_{cam}] \cdot M_{model} \cdot P_{3D\_model}$$

- ▶  $P_{3D\_model}$ : Ein Eckpunkt des virtuellen Würfels.
- ▶  $M_{model}$ : Schiebt den Würfel an die richtige Stelle in der Welt.
- ▶  $[R_{cam}|t_{cam}]$ : Die Extrinsics (das liefert das AR-Tracking in Echtzeit!).
- ▶  $K$ : Die Intrinsics (Brennweite/Zentrum, einmalig kalibriert).
- ▶  $p_{pixel}$ : Der finale 2D-Punkt auf dem Handy-Display.

Fazit: AR ist im Kern das ständige Aktualisieren der Extrinsics-Matrix und das Anwenden der Pinhole-Projektion auf virtuelle Punkte.

## Frameworks - WO kann ich spielen?

| Framework             | Hersteller | Besonderheiten                                                                                                       | Zielplattform               |
|-----------------------|------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------|
| ARCore                | Google     | Standard für Android. Starkes Plane Detection (Boden/Wände).                                                         | Android (Milliarden Geräte) |
| ARKit                 | Apple      | Extrem präzise auf iOS. Nutzt den LiDAR-Sensor im iPad/iPhone Pro für sofortiges 3D-Meshing.                         | iOS (iPhone/iPad)           |
| Vuforia               | PTC        | Industriestandard. Sehr stark bei Model Targets (Erkennen von physischen Objekten wie Motoren anhand von CAD-Daten). | Cross-Platform (Unity)      |
| Unity (AR Foundation) | Unity Tech | Ein Wrapper, der ARKit und ARCore unter einer gemeinsamen API vereint.                                               | Cross-Platform              |